OpenTelemetry Primer

Each concept building on the last

Matthew Reider

OpenTelemetry Primer

© 2026 Matthew Reider. All rights reserved.

Contents

1. Telemetry
2. Signals
3. Metrics
4. Metric Types
5. Outliers
6. Traces
7. Spans
8. Logs
9. Attributes
10. Resources
11. Semantic Conventions
12. Instrumentation
13. Exporters
14. OTLP
15. The Collector
16. Pipelines
17. Deployment Modes
18. Trace Context
19. Baggage
20. Sampling
21. Trace-Aware Routing
22. Events
23. Span Links
24. Correlation
25. Log Correlation
26. Exemplars
27. Service Maps

28. Resource Detectors
29. Collector Transforms
30. The Full Picture
31. Profiling

1. Telemetry

A running program can do one of two things when something goes wrong: fail silently, or tell you what happened. Programs that tell you what happened do so by emitting data about their own behavior. This data is called **telemetry**.

Without telemetry, a service is a closed box. You know it received a request and you know what it returned, but you cannot see what happened inside. With telemetry, the service describes its own internal state as it runs.

```
14:32:01  checkout  received request    /checkout
14:32:01  checkout  queried database    12ms
14:32:02  checkout  sent response       201 Created  387ms
```

2. Signals

Telemetry is not one undifferentiated stream. OpenTelemetry organizes it into three kinds of data, each designed to answer a different question. These kinds are called **signals**.

Metric
: A number measured over time. Answers *how much?* or *how often?*

Trace
: A record of one request's path through services. Answers *where did the time go?*

Log
: A record of a specific event. Answers *what exactly happened?*

The rest of this primer explores each signal, then shows how they connect.

3. Metrics

Imagine ten thousand events per second scrolling past. No one can read them all. A **metric** solves this by counting, averaging, or summarizing those events into a single number over a period of time.

Ten thousand requests become "422 per second." A thousand response times become a single number that represents the group. One way to summarize is with **percentiles**. A percentile tells you what percentage of values fell below a given number. The p99 (99th percentile) means 99 out of 100 requests were faster than this value – only the slowest 1% were slower. The p50 is the median, the typical experience. The p95 captures most users.

```
10,241 requests    →  "422 req/s"
 9,712 durations     →  "p50: 45ms, p95: 210ms, p99: 820ms"
   238 failures      →  "error rate: 2.3%"
```

4. Metric Types

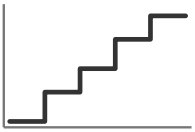
Not all measurements work the same way. OpenTelemetry defines three types of metric, each suited to a different shape of question.

Counter
: A value that only goes up. Total requests served, total bytes sent, total errors. You read it by looking at the rate of increase.

Gauge
: A value that goes up and down. Current memory usage, active connections, queue depth. A snapshot of the present moment.

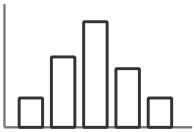
Histogram
: A distribution of values sorted into ranges called buckets. How many requests took 0–100ms? How many took 100–500ms? A histogram is what makes percentiles possible – by counting how

many values land in each bucket, you can calculate the p50, p95, or p99.



counter

gauge



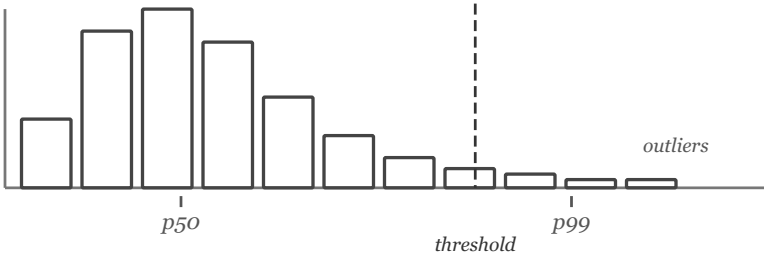
histogram

5. Outliers

An average can hide problems. If 95 requests take 50ms and 5 take 3,000ms, the average is 197ms – a number that describes nobody's actual experience. The fast majority masks the slow few.

Outliers are those slow few. Percentiles expose them where averages cannot. The p99 separates the worst 1% from the rest. When the p99 spikes but the p50 stays flat, you know the problem affects a small subset of requests – a specific customer, a particular query, one overloaded node.

This is why teams set alert thresholds on percentiles rather than averages. An alert that fires when p99 latency exceeds 500ms catches degradation that an average would smooth away.



6. Traces

Metrics can tell you that latency is high, but not which service is responsible. When a user's request passes through five services, which one caused the delay?

A **trace** follows that single request from its entry point to its final response, recording the time spent at each service along the way. If a checkout request passes through gateway, api, orders, and database, the trace shows exactly where the time went:

```
gateway    |-------- 1200ms --------|
  api      |------ 950ms ------|
    orders    |--- 700ms ---|
      db         |- 80ms -|
```

Now you can see that the orders service is the bottleneck.

7. Spans

A trace is made of smaller pieces called **spans**. Each span represents one operation: handling an HTTP request, querying a database, calling another service.

Every span records a start time, a duration, and a status. Spans nest inside each other, forming a tree. The topmost span represents the entire request. Its children represent the steps within it. By reading the tree, you can see which operation took the longest and which one failed.

```
span:
  name       = GET /api/orders
  trace_id   = 4bf92f3577b34da6
  span_id    = 00f067aa0ba902b7
  parent_id  = a3ce929d0e0e4736
  start      = 14:32:01.234
  duration   = 200ms
  status     = OK
```

8. Logs

Metrics tell you *that* something is wrong. Traces show you *where*. Logs explain *why*.

A **log record** captures a single event at a specific moment: an error message, a stack trace, the exact input value that caused a failure. Each record carries a timestamp, a severity level (info, warn, error), and a message body. Where metrics summarize and traces narrate, logs preserve the raw detail.

```
14:32:01 ERROR  checkout-svc
  "Connection pool exhausted: max connections reached"
  exception.type  = PostgresError
  db.statement    = SELECT * FROM orders WHERE id = ?
```

Adding Meaning

9. Attributes

A span that says "HTTP request, 200ms" is vague. A span that says "GET /api/orders, 200ms, status=500, user_id=42" is useful.

The difference is **attributes**: key-value pairs attached to any signal. They provide the context that makes telemetry worth reading – the HTTP method, the endpoint, the response status, the customer tier. Attributes can be attached to spans, metrics, and logs alike.

```
http.request.method  = GET
http.route           = /api/orders
http.response.status = 500
user.id              = 42
cart.item_count      = 3
```

10. Resources

Attributes describe what happened. **Resources** describe who reported it.

A resource is a set of attributes that identify the source of telemetry: the service name, the host, the version, the deployment environment. When multiple services report the same metric, the resource is what distinguishes them.

```
service.name     = checkout-svc
service.version  = 2.4.1
deployment.env   = production
host.name        = prod-checkout-7a3b
```

11. Semantic Conventions

If one team names an attribute *http.status* and another names the same thing *response_code*, no single query can find both.

Semantic conventions are OpenTelemetry's shared naming rules. They define standard names for common concepts – *http.request.method*, *http.response.status_code*, *db.system* – so that data from every team and every service can be queried in the same way.

```
Team A:      http.status             = 200
Team B:      response_code           = 200
Team C:      statusCode              = 200

Convention:  http.response.status_code = 200
```

Producing Telemetry

12. Instrumentation

Code does not produce telemetry on its own. **Instrumentation** is the code that creates spans, records metrics, and emits logs.

OpenTelemetry offers two approaches. **Automatic instrumentation** hooks into common libraries and frameworks – HTTP servers, database drivers, messaging clients – to produce telemetry without code changes. **Manual instrumentation** uses the OpenTelemetry API directly, for custom business logic that libraries cannot cover.

```
# automatic: wraps known libraries, no code changes
opentelemetry-instrument python app.py

# manual: you create spans in your own code
with tracer.start_as_current_span("process-payment"):
    charge(card, amount)
```

13. Exporters

Instrumentation creates telemetry inside a running process. An **exporter** sends it out.

The exporter serializes spans, metrics, and logs into a structured format and transmits them to a destination – a collector, a backend, or a file. It is the boundary between your application and the rest of the telemetry pipeline.

```
your application
  │  spans, metrics, logs
  ↓
exporter
  │  serialize → OTLP
  ↓
collector or backend
```

14. OTLP

Exporters need a format to send data in. **OTLP** (OpenTelemetry Protocol) is the standard wire format for all three signals.

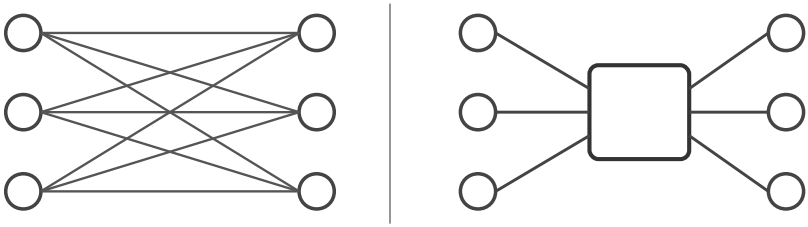
It can be carried over gRPC or HTTP, encoded as binary protobuf or JSON. Because it is a shared standard, any tool that speaks OTLP can receive telemetry from any application that speaks OTLP, regardless of programming language or vendor.

```
POST /v1/traces HTTP/1.1
Content-Type: application/json

{ "resourceSpans": [{
    "resource": { "attributes": [
      { "key": "service.name", "value": "checkout-svc" }
    ]},
    "scopeSpans": [{ "spans": [{ ... }] }]
}]}
```

15. The Collector

Without a central hub, every service must send telemetry directly to every backend it needs to reach. Ten services and three backends means thirty connections to manage.

The **OpenTelemetry Collector** sits between services and backends. Services send telemetry to the Collector. The Collector processes it and forwards it to the right destinations. The number of connections drops from *services* × *backends* to *services* + *backends*.



16. Pipelines

The Collector is configured through **pipelines**, defined in YAML. Each pipeline has three stages:

Receivers
: Accept incoming telemetry. A receiver might listen for OTLP, scrape Prometheus endpoints, or accept Jaeger spans.

Processors
: Transform telemetry in flight – batch it, filter it, enrich it, sample it.

Exporters
: Send the processed data to one or more backends.

Each signal type gets its own pipeline, so traces, metrics, and logs can be handled differently.

17. Deployment Modes

The Collector is a program – it needs to run somewhere. In most environments, that means Kubernetes. A few terms from that world are needed here.

Pod
: One or more containers running together. Each service instance runs in a pod.

Node
: A machine that runs pods. A cluster has many nodes.

DaemonSet
: A Kubernetes resource that automatically runs one pod on every node.

Sidecar
: An extra container that runs alongside your application inside the same pod.

With these terms, there are three ways to deploy the Collector.

DaemonSet
: One Collector per node. Applications send to localhost, so traffic never crosses the network at the collection layer. Simple and efficient, but each Collector only sees telemetry from its own node.

Sidecar

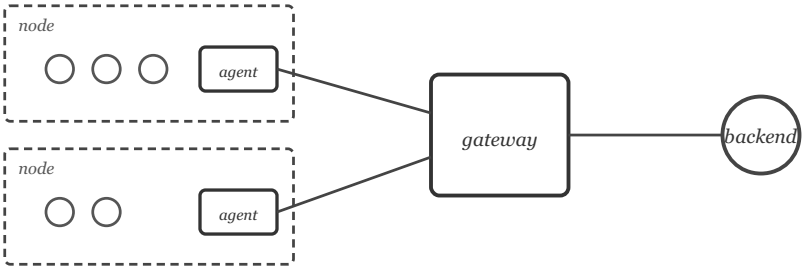
One Collector per pod. Each service gets its own Collector with its own configuration. Useful when teams need different processing rules or strict resource isolation. The tradeoff is overhead – a Collector for every pod is expensive.

Gateway

A centralized deployment – multiple replicas behind a load balancer. Receives telemetry from DaemonSets or sidecars. Handles heavier processing that requires seeing data from many services at once.

In practice, these modes are layered. DaemonSets or sidecars handle lightweight work – batching, resource detection, basic enrichment – and forward to a gateway tier for the expensive processing. The gateway exports to the backend.

```
services → agent (per node)  → gateway (centralized) → backend
               batch, enrich      sample, route, aggregate
```



Crossing Boundaries

18. Trace Context

A trace follows a request across services. But when a request leaves one service and arrives at another, how does the second service know they belong to the same trace?

Through a header. The W3C **traceparent** header carries the trace ID, the current span ID, and trace flags. Every service reads this header, continues the trace, and passes it to the next service it calls.

```
traceparent: 00-4bf92f3577b34da6a3ce929d0e0e4736-00f067aa0ba902b7-01
                ||                ||                  ||          ||
             version          trace-id              span-id    flags
```

19. Baggage

Trace context carries telemetry identity – the trace ID and span ID. **Baggage** uses the same propagation mechanism to carry application data.

A frontend service might set *user.tier=premium* in baggage. Every downstream service receives it automatically, without extra database lookups or API calls. Baggage values can then be added as attributes to spans, making business context available throughout the request.

```
baggage: user.tier=premium,region=us-east-1

frontend  →  api  →  orders  →  database
  (set)     (read)   (read)     (read)
```

Baggage travels with every request. Keep values small and remember that all downstream services can read them.

20. Sampling

At low traffic, you can keep every trace. At high traffic, storing everything becomes too expensive. **Sampling** is the practice of choosing which traces to keep.

Head sampling

Decides at the start of a trace. For example, keep 10% at random. Simple, but blind to whether the trace will be interesting.

Tail sampling

Decides after a trace completes. Keep errors and slow requests; drop routine successes. Smarter, but only works at the gateway tier – a DaemonSet or sidecar sees spans from just one service, not the complete trace.

21. Trace-Aware Routing

Tail sampling requires the complete trace – every span, from every service – at a single gateway replica. But when a trace has twelve spans arriving from five different DaemonSets, a normal load balancer scatters them across replicas. No replica sees the whole trace, and sampling breaks.

The solution is **trace-aware routing**. Instead of round-robin load balancing, each DaemonSet hashes the trace ID to choose which gateway replica receives the span. All spans sharing a trace ID are routed to the same replica, giving it the complete picture.

```
exporters:
  loadbalancing:
    protocol:
      otlp:
        endpoint: gateway-headless:4317
    resolver:
      dns:
        hostname: gateway-headless
```

The **loadbalancingexporter** uses a headless Kubernetes Service, which returns individual pod IPs rather than a single cluster IP. The exporter discovers all gateway pods and applies consistent hashing – so when a pod is added or removed, only a small fraction of traces shift to a new replica.

22. Events

Sometimes you want to record that something happened inside a span without creating a child span for it. An **event** is a lightweight, timestamped annotation attached to a span.

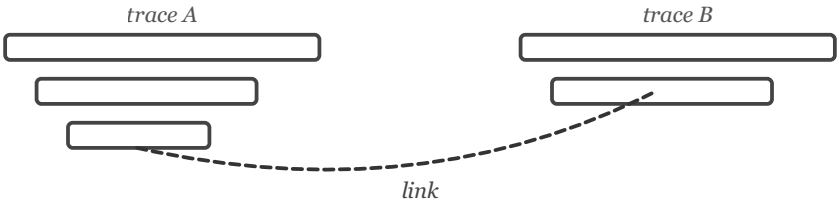
Each event has a name, a timestamp, and optional attributes. Events mark moments like a cache miss, a retry attempt, or a validation failure – things worth noting that do not need their own duration measurement.

```
span: process-order  (340ms)
  events:
    t=28ms   cache.miss     key=inventory:42
    t=155ms  retry          attempt=2
    t=312ms  payment.ok     provider=stripe
```

23. Span Links

Spans within a trace are connected by parent-child relationships. But some connections cross traces entirely. A queue consumer processes a message that was produced in a different trace. A batch job aggregates work from many traces.

Span links express these relationships. They say "this span is related to that span" without changing either trace's structure. The two traces remain independent, but the connection between them is recorded.



24. Correlation

Each signal answers its own question. **Correlation** is the practice of connecting them so you can move between questions in a single investigation.

The trace ID is the shared thread. A metric can reference a trace ID through an exemplar. A log record can carry the trace ID and span ID of the operation that produced it. A span already has its trace ID. With this shared identifier, you can navigate from an alert to a request to an error message – across all three signals.

```
metric:  error_rate = 5.2%            ┐
trace:   checkout → orders → db       ├── trace_id = 4bf92f...
log:     "pool exhausted"             ┘
```

25. Log Correlation

Correlation between logs and traces can be automatic. An OpenTelemetry **log appender** injects the active trace ID and span ID into every log record emitted during a traced operation.

Once injected, every log line is navigable: find a log, jump to its trace. View a span, see every log it produced. The signals stop being separate databases and become one connected record.

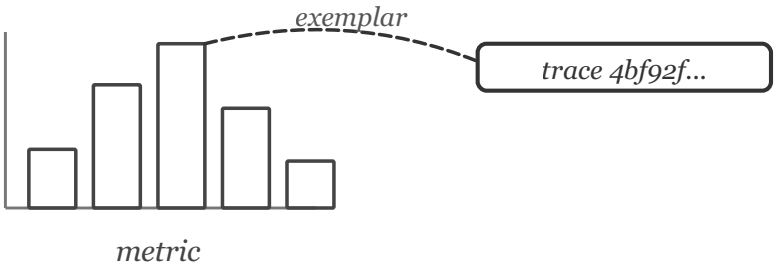
```
before:  14:32:01 ERROR "Connection pool exhausted"

after:   14:32:01 ERROR "Connection pool exhausted"
           trace_id = 4bf92f3577b34da6
           span_id  = 00f067aa0ba902b7
```

26. Exemplars

Metrics aggregate – that is their strength. A histogram can show that some requests took over one second, but not which ones.

An **exemplar** is a link from a metric data point to the specific trace that produced it. The metric says "something was slow." The

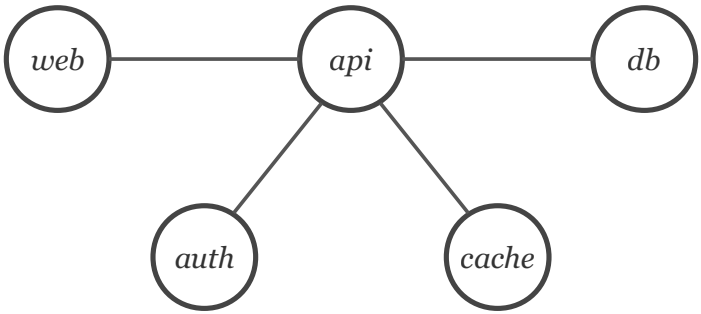
exemplar says "here is one of the slow requests – follow this trace ID to see exactly what happened."



27. Service Maps

Nobody maintains an architecture diagram by hand. The traces maintain it for you.

Every span records which service made a call and which service received it. Analyze enough traces and you get a live **service map**: a topology of your system showing which services exist, how they connect, and where the dependencies run. It is always current because it is drawn from real traffic.



28. Resource Detectors

Resource detectors run at startup and automatically discover the attributes that describe the environment: host name, cloud provider, Kubernetes namespace, OS type, container ID.

The only resource attribute that teams typically set by hand is *service.name*. Everything else – where the code is running, on what hardware, in which region – can be detected from the runtime environment.

```
service.name   = checkout-svc         ← set by you
cloud.provider = aws                  ← detected
cloud.region   = us-east-1            ← detected
k8s.namespace  = production           ← detected
host.type      = m5.xlarge            ← detected
container.id   = a1b2c3d4e5f6         ← detected
```

29. Collector Transforms

The Collector can reshape telemetry in flight without redeploying any application. Its **transform processor** modifies data as it passes through a pipeline:

Delete attributes
: Strip email addresses or user names before data leaves your infrastructure.

Add attributes
: Tag all telemetry with a region or team name.

Filter spans
: Drop health-check noise that adds volume without insight.

Rename attributes
: Normalize names to match semantic conventions without changing application code.

30. The Full Picture

Real incidents require all three signals working together.

A metric triggers the alert: error rate spiked at 14:32. A trace narrows the scope: the payment service is timing out on calls to the database. A log explains why: "connection pool exhausted – max connections reached."

No single signal is sufficient. Metrics establish *when* and *what*. Traces pinpoint *where*. Logs reveal *why*. Correlation lets you move between them. Together, they reach root cause.

```
1. metric    error_rate > 5%     alert fires at 14:32
       ↓
2. trace     checkout → payment → db    payment: 3200ms timeout
       ↓
3. log       "connection pool exhausted: max connections reached"
       ↓
   root cause: database connection limit needs increasing
```

31. Profiling

Traces show where time is spent across services. **Profiling** shows where time is spent inside a single service – at the function level – by capturing the actual stack frames executing on the CPU.

OpenTelemetry's profiling signal links profiles to traces through span context. A slow span does not just tell you the operation took 700ms; it connects to a flamegraph showing which functions consumed that time.

```
processRequest ──────────────────────── 700ms ──────
├─ validateInput ── 20ms ──┤
├─ queryDatabase ──────────────── 580ms ──────────┤
│  ├─ getConnection ─── 490ms ──────────┤  ← slow
│  └─ executeSQL ── 90ms ──┤
└─ formatResponse ── 15ms ──┤
```

Profiling is an emerging signal in OpenTelemetry. The specification is under active development.

About the Author

Matthew Reider has spent more than twenty years in software, much of it at the intersection of open source communities and infrastructure. He found his footing in the Ruby on Rails community at Engine Yard, helped grow Cloud Foundry into one of the first enterprise platform-as-a-service projects, and now works at Dynatrace alongside the Kubernetes and OpenTelemetry communities. He lives in Vienna, Austria with his wife, kids, cats, and dog.